Project Proposal: Smartphone Gait-Based Owner Verification

Author Name

April 23, 2026

1 1. Preliminary Domain Knowledge

During our preliminary research, we were especially interested in machine learning projects that make use of data already collected by everyday devices. Smartphones stood out as a strong example because they continuously record information from built-in sensors such as accelerometers and gyroscopes. This suggests that phones may support useful security or health-related applications without requiring extra hardware.

For our project, we want to apply this idea to a practical security problem: determining whether a phone is still in its owner's control while the person carrying it is walking. This matters because theft or unauthorized access does not always happen in an obvious way. Someone might take a phone and walk away with it, but it could also be picked up briefly for a nearby action such as making a payment and then returned. In either case, detecting only the exact moment the phone is taken is usually not enough.

Our main idea is that a person's gait may act as a behavioral fingerprint. A phone carried while walking records repeated motion patterns, and we believe these signals may contain user-specific information such as stride timing, acceleration structure, and frequency patterns. We found a previous study and repository that approached a related problem as a multiclass classification task, which suggests that different people may produce distinguishable motion signatures with high predictive accuracy.

Based on this, our project aims to model walking sequences from smartphone accelerometer and gyroscope data and map them into an embedding space. We then plan to use cosine similarity or another comparison method to estimate whether a new walking segment is likely to come from the phone's owner or from someone else. In other words, the machine learning task is owner-versus-non-owner verification based on gait-related sensor data collected during walking.

2 2. Preliminary Data Exploration

- Which features are in your dataset?
- How are your features distributed?

- Are some of your features correlated with each other or with the target label?
- How many classes are there and are they balanced?
- Does some of the information here contradict or confirm prior domain knowledge?
- Are there any missing values, or outliers in your data?
- Which plots will you include to show what your dataset is all about?
- Don't forget to interpret your plots.
- **Required:** 1 dimensionality reduced plot, 2 other plots.

The dataset we found on GitHub contains 3-axis gyroscope readings (GYR) and 3-axis accelerometer readings (ACC) over time. This gives us the channels ACC_x, ACC_y, ACC_z, GYR_x, GYR_y, and GYR_z, each sampled at 50 Hz. Since the task we are framing is a yes/no verification problem, the class balance can be controlled during training by pairing each positive example with a negative comparison example.

There is naturally correlation in the data because these channels represent different axes in the same physical space, but this is not necessarily a limitation. Instead, the channels are complementary because together they capture spatial information about the motion of a step and its effect on acceleration and rotation. This generally supports our original expectations rather than contradicting them.

3 3. Proposed Preprocessing

- What preprocessing steps will you be doing?
- Will you use normalization?
- Will you use feature extraction?
- Will you use dimensionality reduction or feature selection?
- Will you use value imputation for NaNs or other bad data?
- How will you split your data?
- How will you deal with class imbalance?
- Is there anything very specific to your topic that needs preprocessing?
- Which methods will you be using and why?
- Where possible, how do these choices connect to your findings from Questions 1 and 2?
- Make sure it is very clear to the reader what will happen.
- **Required:** The proposed preprocessing steps.

- **Recommended:** 1 draw.io flowchart that shows the preprocessing steps and their order.

We plan to:

- filter out obvious sensor anomalies or corrupted values if they appear;
- split the data by participant rather than randomly, so that the same person does not appear in both training and test data;
- avoid the original paper's percentage-based split across each participant and instead enforce subject segregation;
- apply per-channel z-score normalization using statistics computed only on the training data;
- use subject-group K-fold cross-validation by dividing the 180 subjects into groups and folding by subject group;
- train on most subject groups, then for each test participant use a small amount of data to build the owner embedding and reserve the rest for testing;
- compare two preprocessing directions: using raw windows directly for deep learning, or doing feature engineering first;
- consider time-domain features such as mean, standard deviation, energy, turning points, and range, as well as frequency-domain features such as FFT band energy, dominant frequency, and spectral entropy;
- clip extreme values if we observe unrealistic sensor readings, although we have not seen obvious cases yet;
- consider time warping as a data augmentation method, although some temporal alignment is already handled through interpolation;
- decide whether to follow the paper's compound-acceleration peak detection and two-step segmentation, or instead use broader movement windows if they capture more useful information; and
- select random adversarial or impostor subjects for each point so that each example is explicitly compared against someone it is not.

4 4. Proposed Model(s) and Baseline

- Which simple baseline model will you compare your solution to?
- Which "proper" model(s) will you use to get a well-performing ML system?
- Why are you choosing these models?
- How will you train your model(s)?
- What hyperparameter tuning strategy will you implement?

- Which hyperparameters will you tune?
- If you use an SVM, what kernel function will you choose and why?
- **Required:** At least 1 baseline and at least 1 "full" model.

We plan to:

- **Baseline:** FFT-based features with an SVM, or a similarly simple classical model.
- **Deep learning model:** still to be decided, but it will likely be trained on raw or lightly processed windows.
- **Loss:** triplet loss.
- **Model comparison:** accuracy, precision, recall, F1 score, confusion matrix, and K-fold comparison.
- **Hyperparameter search:** grid search.
- **Regularization techniques:** dropout, L2 regularization, early stopping based on F1, batch normalization, layer normalization, and Adam for optimization.
- **Hyperparameters to tune:** dropout, L2 strength, layer count, and node count.
- **Deployment goal:** compress the model to run on a phone through pruning, compression, quantization, or related methods.
- **Application goal:** build a web or phone app that records gyroscopic data live, feeds it to the model, and returns live predictions.

5 5. Proposed Evaluation

- How will you assess your model(s) in the end?
- Which quantitative metrics will you use for predictive performance?
- Will you include accuracy, F1, AUROC, MAE, MSE, or other metrics?
- Will you use things that reflect system behavior, such as confusion matrices?
- How will you evaluate beyond predictive performance?
- Will you consider train cost, inference cost, model size, or social biases?
- How will you detect overfitting or underfitting?
- **Required:** At least one quantitative metric for predictive performance.
- **Required:** At least one other evaluation.

- **Metrics:** accuracy, precision, recall, F1 score, confusion matrix, K-fold comparison, and AUROC.
- **Evaluation of learned patterns:** for FFT-based features, we can inspect which frequencies are most useful for distinguishing between different pocket placements. We can visualize these features with a bar plot or heatmap to better understand what contributes to the model's predictions.
- **Embedding visualization:** we want to visualize the embedding space to better understand what the model is using to separate users. If possible, we may also use methods similar to Grad-CAM or other interpretability tools.
- **Evaluation with new data:** we will record new data from a few volunteers who were not part of the original dataset. They will carry the phone in different pockets while walking, and we will use this data to evaluate the trained model. This should give us a better understanding of how well the model generalizes to new people and more realistic scenarios.

6 6. Model Usage

- How could your model be used?
- Why could your model be used?
- By whom could your model be used?
- What needs to happen so users can make a request and get a usable response?
- What happens before your model runs?
- What happens after your model makes a prediction?
- How will you turn your model's prediction into a usable answer for the user?
- **Required:** At least one user group and one use case.
- **Required:** Something that happens before and something that happens after your model.

Our model could be used for:

- theft detection;
- suspicious activity detection;
- purchase confirmation based on recent motion behavior; and
- other lightweight security checks that require confirmation of identity.

Why: the system is relatively small, can run on-device, and has access to live motion data.

Who could use it: anyone with a smartphone, especially users who want an extra passive layer of security.

A user could download the app or access an API-based web app. If we can make the model small enough, we may be able to deploy it directly in the browser or as a lightweight local download. There would first be a short enrollment period where the user walks for a few minutes to create an identification embedding. After that, live data would be streamed in, interpolated and resampled to handle jitter, and then compared through cosine similarity or another comparison method against the known owner embedding.

Most of the time the system would remain idle. If it makes a prediction that suggests a possible threat, the app could trigger an alert, message, or notification on another device.

The final output is essentially a yes/no decision, so the user-facing result would likely be a notification on another device. To support that, we would need SMS, email, bot-based messaging, or a server that can receive the signal and send a ping to another device.

7 7. Risk Assessment

- What are the potential risks related to your project?
- What issues may you expect during development?
- How would you circumvent this?
- How likely is this project to be successful?
- What are societal or user risks when your model is deployed publicly?
- **Required:** At least one threat to your project development and an estimate of success.
- **Required:** A consideration of possible risks of having your model deployed.

One major risk is the use of live streaming data, since this is sensitive information. For that reason, we would prefer to keep the system on-device rather than stream sensor data to a server.

We may also face a trade-off between model size and accuracy, and we may need to learn deployment techniques that are specific to this use case. To prepare for that, we want to look early at projects such as the web `fake beer` drinking app to understand how live streaming sensor data is handled, and we may also need to explore mobile development.

The project is likely to be successful in terms of model accuracy, since the task has already been shown to be possible with high accuracy. What is less certain is how feasible the approach will be once compression and deployment constraints are added.

The main societal risk is misuse: the system could be used to detect, follow, or identify someone through their embedding, which would effectively leak biometric information. There are also user

risks because this system would be an added security layer, and users might blame the model if it fails in a sensitive situation. In a real deployment, the model would need to achieve a very high level of reliability before it could be trusted.

8 8. Individual Learning Outcomes

- What would each group member like to learn beyond the course learning outcomes, and why?
- **Required:** One learning outcome, written as a full sentence, per group member, with reasoning.
- Possible topics: project management and steering, IDEs, writing tests, Transformers, clean code, or something else.

Will: I would like to learn more about model compression as a general practice and about the nuance of deploying such a model. This connects to my interest in neuromorphic computing, because smaller neural networks are easier to simulate in that setting and can provide useful insight into how these ideas might be implemented in circuitry.

Haukur:

Tijje:

Efe:

9 9. Relation to Grading Specifications

- How does your project meet the minimum requirements?
- What is your argument for novelty?
- What is your preprocessing plan?
- Which hyperparameters might you optimize?
- Which things do you intend to do to boost your grade?
- Which things do you intend to do to further boost your grade?

10 10. Timeline

- What is your rough outline of how much time you plan to allocate to different parts of the project?
- Which tasks can be parallelized across group members?
- Keep in mind that the actual project may deviate from this plan as some parts need more or less time than anticipated.